# Лекция 14. Масштабирование, стоимость и обзор LLMOps

Версия: 2026–2027 учебный год

Продолжительность: 2 академических часа

Связанные работы: ЛР14 «Responsible ML», ЛР15 «Нагрузочный протокол и масштабирование»

## 1. Цель лекции

Научить студентов измерять производительность ML-сервиса, отличать симптом от bottleneck и принимать решение о масштабировании с учётом SLO, восстановления, стоимости и рисков. Показать, как знакомые MLOps-инварианты расширяются на LLMOps без обязательного вызова внешней модели.

## 2. Результаты обучения

Студент сможет:

• определить workload model и воспроизводимый нагрузочный протокол;

• интерпретировать throughput, error rate, p50, p95, max и saturation;

• объяснить warm-up, burst, soak и recovery scenario;

• обнаружить client bottleneck и coordinated omission;

• различать вертикальное и горизонтальное масштабирование;

• связать Kubernetes requests, limits, probes и HPA;

• оценить стоимость baseline и candidate capacity;

• перенести versioning, tracing, evaluation и security в LLMOps.

## 3. Место в сквозном проекте

ЛР11 добавила метрики сервиса, ЛР12 — качество модели, ЛР14 — ограничения ответственного выпуска. Теперь команда проверяет, выдерживает ли конкретная версия image и модели заявленный профиль и возвращается ли сервис в готовность после контролируемого нарушения.

## 4. Производительность — свойство сценария

Число «100 запросов в секунду» бессодержательно без payload, concurrency/arrival rate, среды, длительности, warm-up, timeout и критерия успеха. Сравнивать можно только прогоны с одинаковым протоколом или с явно объяснённым изменением одного фактора.

## 5. Workload model

Закрытая модель держит заданное число одновременных клиентов: новый запрос начинается после ответа. Открытая модель задаёт arrival rate независимо от ответа. Первая проще для учебной лаборатории, но может сама уменьшить нагрузку при замедлении сервиса. Этот эффект нужно назвать ограничением.

## 6. Профили нагрузки

• warm-up — загрузка модели, соединений и cache; его метрики не смешиваются со steady;

• steady — ожидаемая длительная нагрузка;

• burst — кратковременный пик;

• soak — длительный прогон для утечек и накопления очереди;

• recovery — восстановление readiness после сбоя или restart.

В двухчасовой ЛР обязательны steady, burst и короткий recovery drill; soak остаётся расширением.

## 7. Throughput и concurrency

Throughput — завершённые запросы за секунду. Concurrency — число запросов в работе. При стабильной системе закон Литтла связывает среднее число задач N, интенсивность λ и время W: N = λW. Это проверка здравого смысла, а не замена измерению распределения latency.

## 8. Latency percentiles

p50 описывает типичный ответ, p95 — хвост для большинства пользователей, max помогает увидеть аномалию. Среднее скрывает тяжёлый хвост. В проекте используется nearest-rank: сортируем n значений и берём элемент с индексом `ceil(q*n)`. Метод фиксируется, чтобы разные инструменты не дали спор.

## 9. Error rate

Server errors, timeouts и connection failures относятся к capacity/reliability. Намеренные 4xx проверяют контракт клиента и анализируются отдельно. Если смешать 422 с 5xx, можно объявить здоровый сервис перегруженным.

## 10. Размер выборки и повторяемость

Двадцать запросов — только учебный минимум для механики. В реальном решении нужны повторные прогоны, доверительные интервалы и контроль фоновой нагрузки. Команда сохраняет сырые latency, а не только готовый p95, и повторяет сценарий не менее трёх раз.

## 11. Client bottleneck

Генератор может ограничивать throughput раньше сервиса: один поток, медленный DNS, лимит сокетов, CPU ноутбука или сериализация. Нагрузочный клиент и сервис наблюдаются отдельно. Рост latency клиента без saturation сервиса требует проверки генератора и сети.

## 12. Coordinated omission

В закрытой модели медленный ответ откладывает следующий запрос, поэтому худший период получает меньше измерений. Это coordinated omission. Для high-stakes теста используют arrival-rate генератор или корректируют методику; в ЛР ограничение явно записывается в отчёте.

## 13. Saturation и bottleneck

Наблюдаются CPU, memory, queue/backlog, connection pool, disk/network I/O, garbage collection и число worker. Высокий CPU с ростом p95 отличается от низкого CPU и медленного внешнего I/O. Решение «добавить pods» без локализации bottleneck может увеличить стоимость без эффекта.

## 14. Один фактор за эксперимент

Сначала фиксируют baseline. Затем меняют только workers, replica count, CPU request, batch size или другой фактор. Одновременная смена image, модели, payload и инфраструктуры делает вывод неидентифицируемым.

## 15. Recovery time

Recovery измеряется от контролируемого нарушения до устойчивой готовности, например трёх успешных readiness checks. Один ответ 200 может быть случайным. В отчёте фиксируются команда restart, timestamps, потерянные запросы и причина восстановления.

## 16. Readiness, liveness и startup probes

Readiness убирает pod из Service endpoints, liveness перезапускает зависший контейнер, startup защищает медленный старт от преждевременной liveness. Неверная liveness probe под нагрузкой может создать каскад перезапусков. Эти три сигнала нельзя считать взаимозаменяемыми.

## 17. Requests и limits

CPU/memory requests используются планировщиком и задают базу для CPU utilization HPA. Limits ограничивают использование: CPU может throttling, превышение memory limit — завершить контейнер. Поднимать limit без измерения не означает повысить гарантированную capacity.

## 18. Вертикальное и горизонтальное масштабирование

Вертикальное масштабирование увеличивает ресурсы экземпляра; горизонтальное — число replicas. Горизонтальный подход требует stateless API или внешнего состояния, корректной балансировки и достаточной downstream capacity. Модель, загруженная в память каждого pod, умножает memory cost.

## 19. Horizontal Pod Autoscaler

HPA `autoscaling/v2` меняет replicas по CPU, memory или custom/external metrics. CPU HPA не видит p95 напрямую. Если latency растёт при низком CPU из-за I/O, такой HPA может не помочь. Metrics pipeline, resource requests и readiness являются предпосылками корректного поведения.

## 20. Stabilization и flapping

Без стабилизации шумная метрика заставляет replicas колебаться. В демо быстрый scale-up ограничен политикой, а scale-down использует окно 300 секунд. Медленное уменьшение capacity снижает риск удалить pods сразу после короткого спада.

## 21. PodDisruptionBudget

PDB ограничивает добровольные disruptions, но не гарантирует доступность при crash, исчерпании узла или ошибке приложения. Он дополняет replicas, probes и rolling update. В демо `minAvailable: 1` соответствует учебной конфигурации, но production-значение выводится из SLO.

## 22. Kubernetes в курсе

Deployment, Service, HPA и PDB хранятся как manifests для архитектурного review. Локальный cluster, metrics-server и `kubectl apply` не являются условием допуска. Студент обязан объяснить поля и предпосылки, а не предоставлять скриншот чужого кластера.

## 23. Стоимость capacity

Минимальная оценка: replicas × цена replica-hour × часы. Полная стоимость включает registry, балансировщик, monitoring, storage, egress, idle headroom и инженерное сопровождение. Учебные условные единицы сравнивают варианты, но не выдаются за коммерческое предложение провайдера.

## 24. Decision record

Решение содержит baseline, SLO, evidence, bottleneck hypothesis, альтернативы, candidate capacity, ожидаемый эффект, стоимость, риски, rollback trigger и владельца. Допустимые действия: keep, optimize, scale, limit workload или redesign.

## 25. Почему autoscaling не заменяет capacity planning

HPA реагирует после наблюдения метрики и имеет задержку запуска pod. Он не создаёт мгновенную capacity, не чинит downstream и не знает бизнес-стоимость ошибки. Нужны headroom, admission control, timeout, retry budget и graceful degradation.

## 26. Переход к LLMOps

Генеративное приложение добавляет prompt, system instructions, provider/model, temperature, tools, retrieval corpus и conversation state. Эти элементы становятся версионируемыми зависимостями, подобно коду, данным и модели классического ML.

## 27. Tracing LLM-приложения

Trace связывает запрос, шаги retrieval/tool/model, output, latency, token usage и ошибки. Нельзя безусловно сохранять полный prompt/response: применяется минимизация, маскирование и retention. Trace ID помогает воспроизвести сбой, но не является лицензией на сбор чувствительного текста.

## 28. Eval dataset и scorers

Набор должен включать типовые и критические кейсы: формат, корректность, grounding, отказ, injection, утечка и tool misuse. Scorer может быть детерминированным, основанным на модели-судье или human review. Для judge фиксируются модель, prompt, шкала и обработка разногласий.

## 29. MLflow GenAI

Актуальный MLflow разделяет `mlflow.genai.evaluate()` для GenAI/traces и `mlflow.models.evaluate()` для классического ML. Tracing помогает видеть шаги, latency и token usage. В курсе это обзор: внешний API, ключ и платный вызов не требуются.

## 30. Latency и стоимость LLM

Измеряются time-to-first-token, полная latency, input/output tokens, tool calls, cache hit и стоимость на успешный кейс. Дешёвый ответ, не прошедший critical scorer, не является оптимизацией. Batch, caching и routing меняют одновременно качество, latency и риск устаревшего контекста.

## 31. Демонстрация преподавателя

1. Запустить capacity audit на baseline sample.
2. Увеличить две burst latency и получить `review`.
3. Сравнить throughput, p95 и CPU-график; обсудить bottleneck.
4. Показать probes, requests/limits, HPA stabilization и PDB.
5. Рассчитать стоимость двух и трёх replicas.
6. Открыть LLMOps eval cases и классифицировать deterministic/model/human scorer.

## 32. Вопросы аудитории

1. Почему рост p95 не всегда требует нового pod?
2. Чем concurrency отличается от arrival rate?
3. Как coordinated omission искажает результат?
4. Почему readiness не должна перезапускать контейнер?
5. Что необходимо CPU HPA для расчёта utilization?
6. Почему PDB не является гарантией availability?
7. Что кроме prompt нужно версионировать в LLMOps?
8. Когда model-as-judge требует human review?

## 33. Связь с ЛР15 и ЛР16

В ЛР15 команда строит профиль, recovery evidence и capacity decision. В ЛР16 эти результаты вместе с model/data governance, registry и observability войдут в итоговый release manifest и защиту.

## 34. Официальные источники

- Kubernetes HPA: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
- Kubernetes resources: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- Kubernetes probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/
- MLflow GenAI tracing: https://mlflow.org/docs/latest/genai/getting-started/tracing
- MLflow GenAI evaluation API: https://mlflow.org/docs/latest/api_reference/python_api/mlflow.genai.html